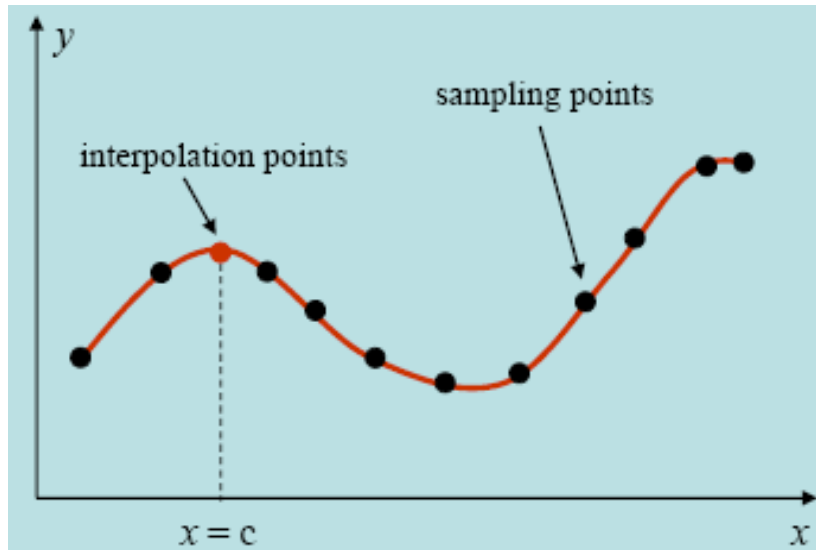


CE 206: Engineering Computation Sessional

Curve-fitting and Interpolation

Curve-fitting and engineering practice

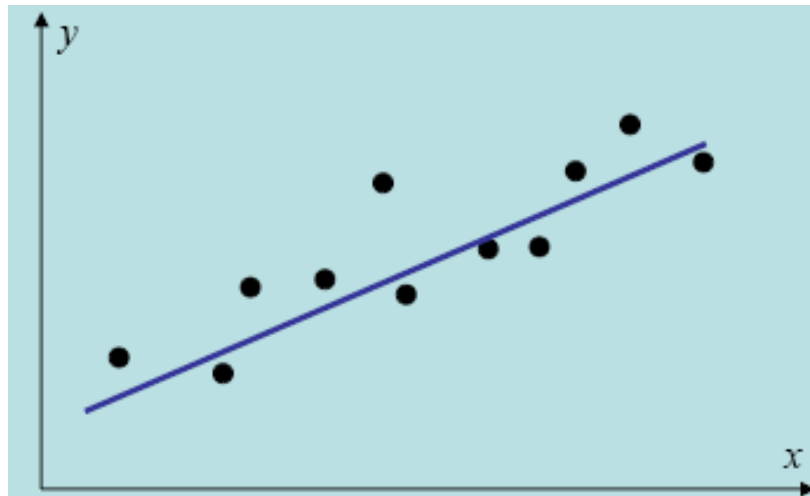


sampling data are acquired at *discrete points*.

- but the values at undefined points are wanted in many engineering applications

Interpolation

- Ex: density of water at different temp.



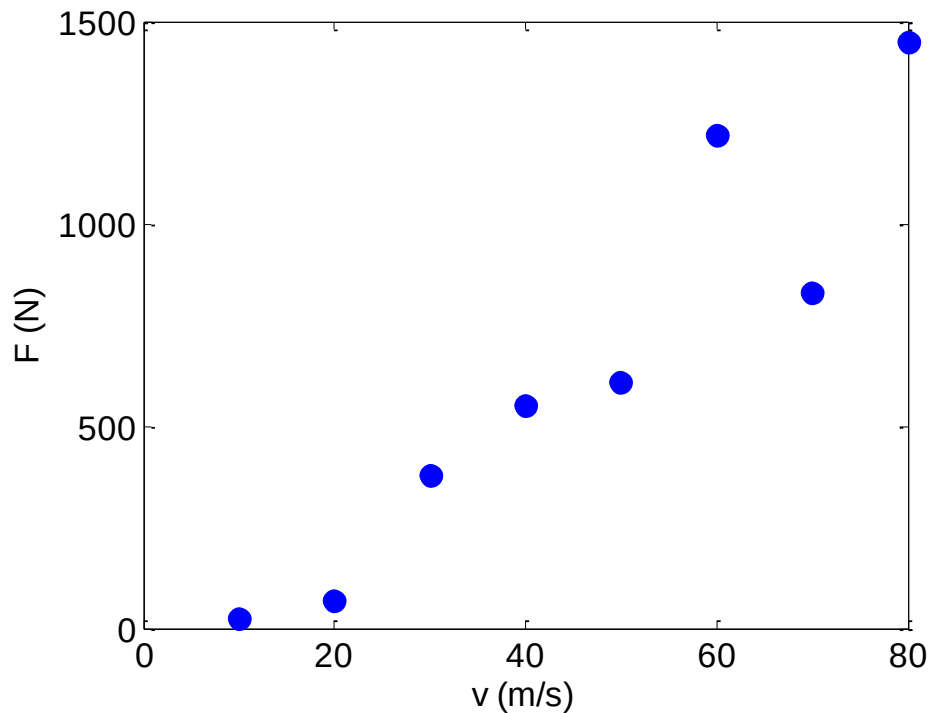
Least-squares Regression

- Purpose is trend analysis or hypothesis testing

Trend of experimental data

Experimental data for force (N) and velocity (m/s) from a wind tunnel experiment

$v, \text{m/s}$	10	20	30	40	50	60	70	80
F, N	25	70	380	550	610	1220	830	1450



Is the relationship
-linear? Which order?

*(Linear or polynomial
regression)*

-nonlinear?

(Nonlinear regression)

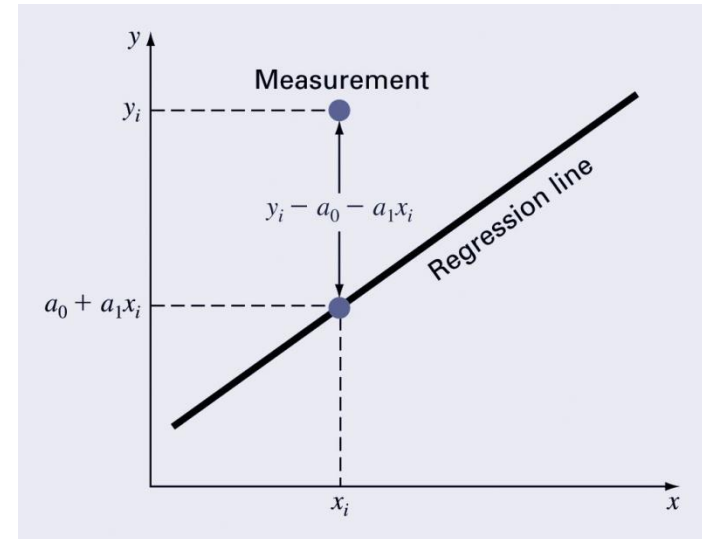
Fitting the data with a straight line

Minimizing the sum of squares of the estimated residuals

$$S_r = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - a_0 - a_1 x_i)^2$$

$$a_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$$

$$a_0 = \bar{y} - a_1 \bar{x}$$



Testing the goodness of fit

$$r^2 = \frac{S_t - S_r}{S_t} \quad \left\{ \begin{array}{l} S_t = \sum_{i=1}^n (y_i - \bar{y})^2 \\ S_r = \sum_{i=1}^n (y_i - a_0 - a_1 x_i)^2 \end{array} \right.$$

(CE 205 lecture slides of details)

A MATLAB code for linear regression

```
function [a, r2] = linregr(x,y)
% input:
%   x = independent variable
%   y = dependent variable
% output:
%   a = vector of slope, a(1), and intercept, a(2)
%   r2 = coefficient of determination
n = length(x);
if length(y)~=n, error('x and y must be same length'); end
x = x(:); y = y(:); % convert to column vectors
sx = sum(x); sy = sum(y);
sx2 = sum(x.*x); sxy = sum(x.*y); sy2 = sum(y.*y);
a(1) = (n*sxy-sx*sy)/(n*sx2-sx^2);
a(2) = sy/n-a(1)*sx/n;
r2 = ((n*sxy-sx*sy)/sqrt(n*sx2-sx^2)/sqrt(n*sy2-sy^2))^2;
% create plot of data and best fit line
xp = linspace(min(x),max(x),2);
yp = a(1)*xp+a(2);
plot(x,y,'o',xp,yp)
grid on
```

Fitting a straight line

```
>> [a, r2] = linregr(v,F)
```

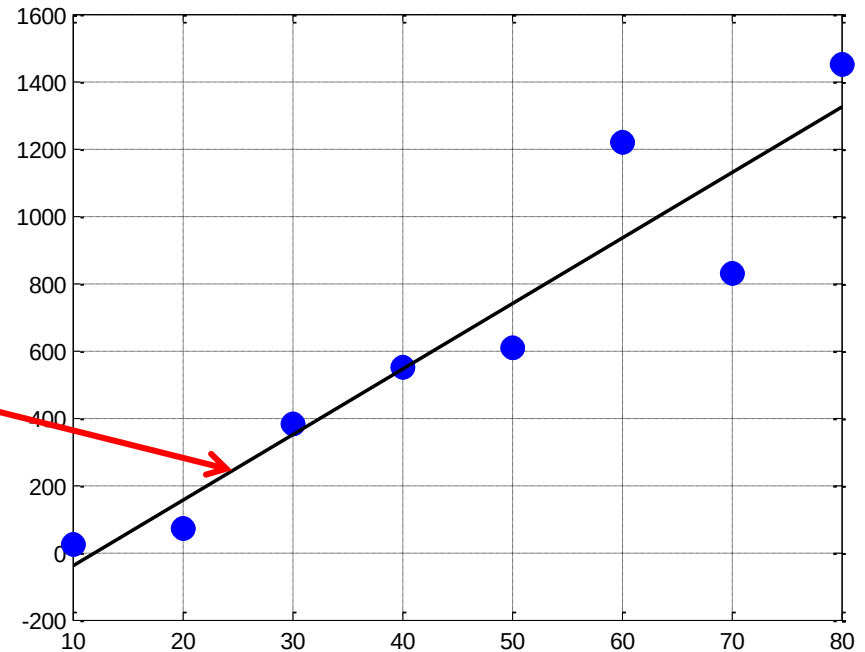
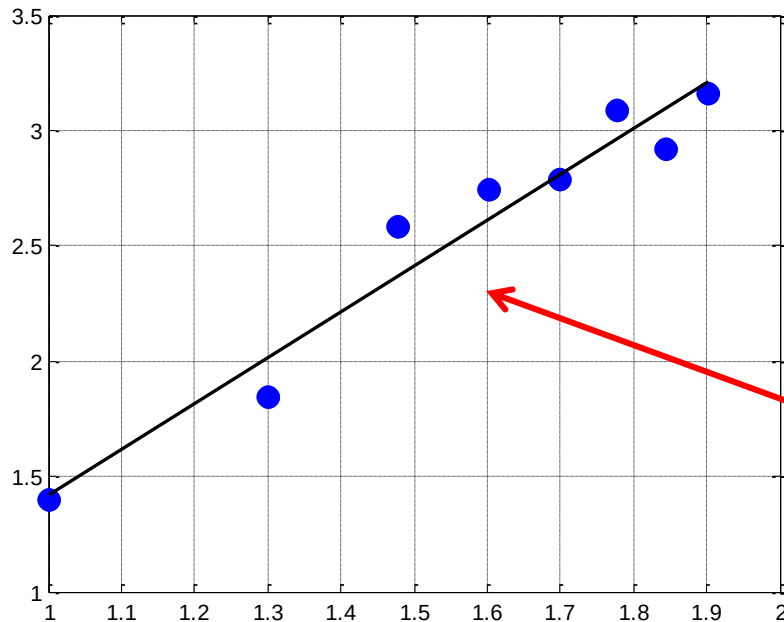
```
a =
```

```
19.4702 -234.2857
```

```
r2 =
```

```
0.8805
```

$$F = -234.2857 + 19.4702v$$



Log-transformed data

```
>> [a, r2] =
```

```
linregr(log10(v), log10(F))
```

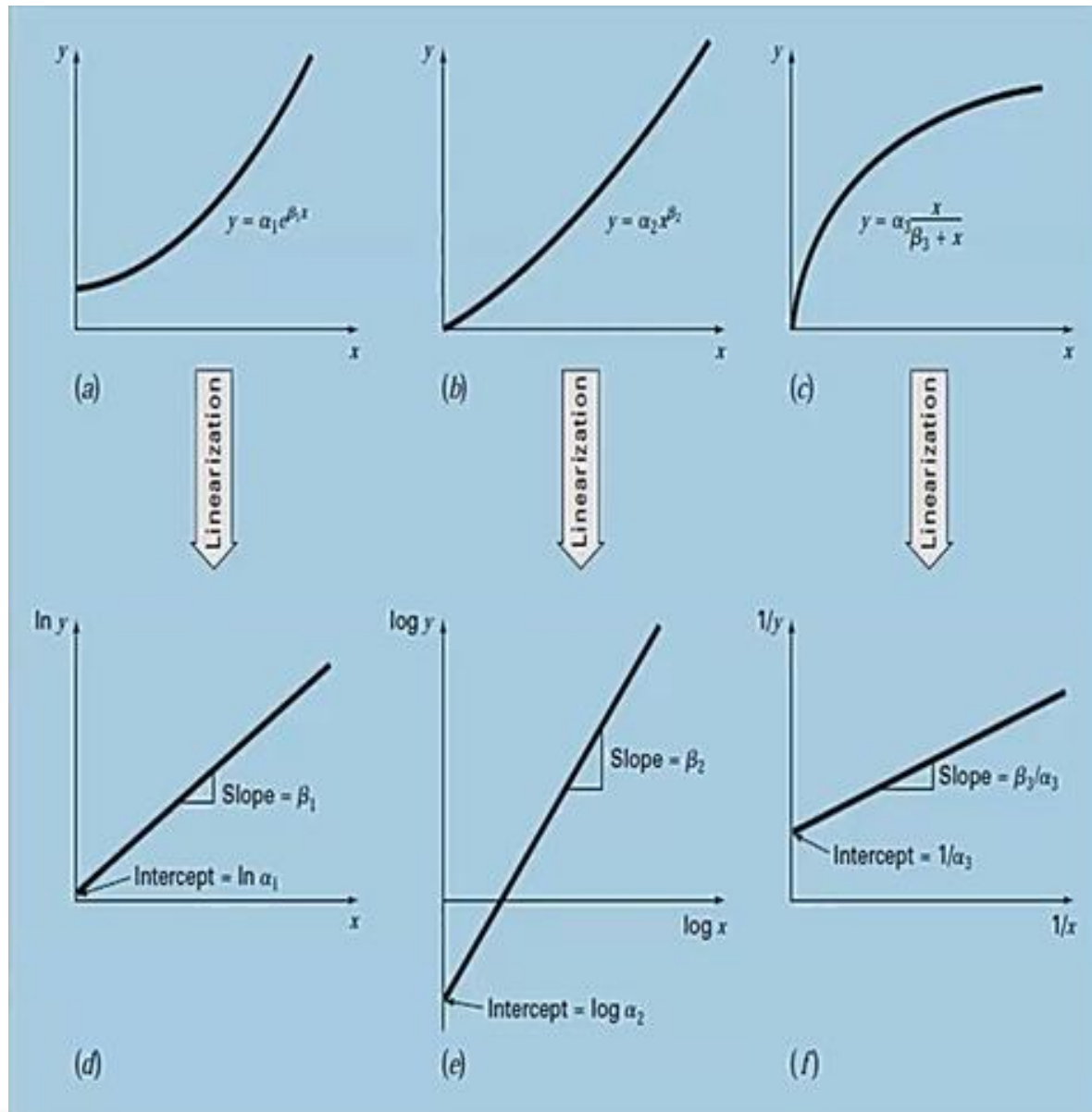
```
a =
```

```
1.9842 -0.5620
```

```
r2 =
```

```
0.9481
```

Fitting a straight line



Variety of linear regressions

To fit a second order polynomial

$$S_r = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - a_0 - a_1 x_i - a_2 x_i^2)^2$$

$$\begin{bmatrix} n & \sum x_i & \sum x_i^2 \\ \sum x_i & \sum x_i^2 & \sum x_i^3 \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 \end{bmatrix} \begin{Bmatrix} a_0 \\ a_1 \\ a_2 \end{Bmatrix} = \begin{Bmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \end{Bmatrix}$$

Higher order polynomials

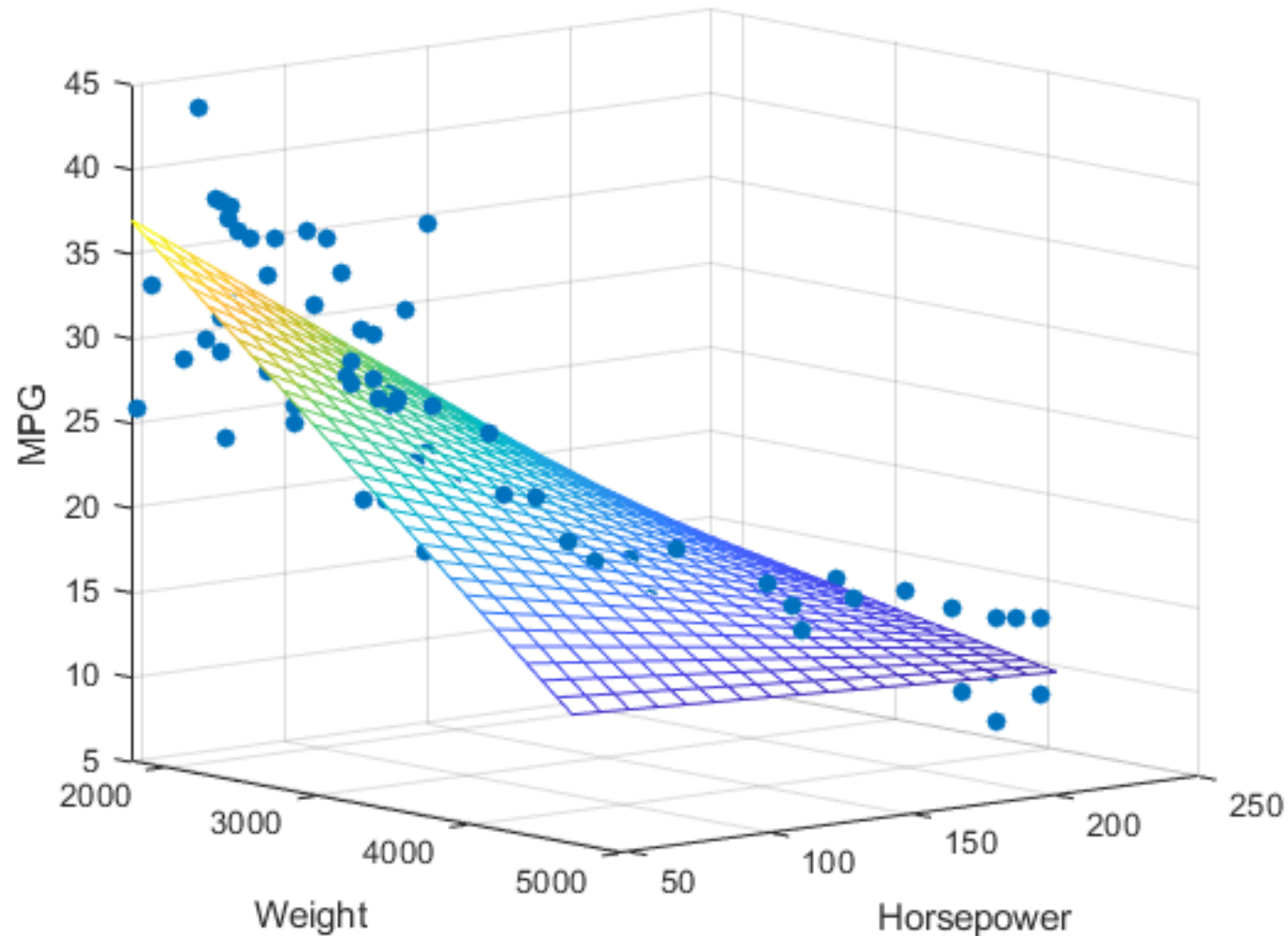
$$S_r = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - a_0 - a_1 x_i - a_2 x_i^2 - \cdots - a_m x_i^m)^2$$

Multiple linear regression

$$y = a_0 + a_1 x_1 + a_2 x_2 + \cdots + a_m x_m$$

$$S_r = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - a_0 - a_1 x_{1,i} - a_2 x_{2,i} - \cdots - a_m x_{m,i})^2$$

Multiple linear regressions



Generalizing least squares

Linear, polynomial, and multiple linear regression all belong to the general linear least-squares model:

$$y = a_0 z_0 + a_1 z_1 + a_2 z_2 + \cdots a_m z_m + e$$

where $z_0, z_1, \dots, z_m$ are a set of $m+1$ *basis functions* and e is the error of the fit

$$\{y\} = [Z]\{a\} + \{e\}$$

$$[Z] = \begin{bmatrix} z_{01} & z_{11} & \cdots & z_{m1} \\ z_{02} & z_{12} & \cdots & z_{m2} \\ \vdots & \vdots & \ddots & \vdots \\ z_{0n} & z_{1n} & \cdots & z_{mn} \end{bmatrix}$$

z_{ji} represents the the value of the j_{th} basis function calculated at the i_{th} point.

Generalizing least squares

$$\{y\} = [Z]\{a\} + \{e\}$$

Applying the condition of least squares

$$S_r = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n \left(y_i - \sum_{j=0}^m a_j z_{ji} \right)^2$$

We get the following:

$$[Z]^T [Z] \{a\} = [Z]^T \{y\}$$

We can solve for the coefficients $\{a\}$ using matrix manipulations

To calculate residuals: $\{y\} - [Z]\{a\}$

Example: general least squares

Fit the wind tunnel data to a 2nd order polynomial

$v, \text{ m/s}$	10	20	30	40	50	60	70	80
$F, \text{ N}$	25	70	380	550	610	1220	830	1450

Create the Z matrix:

```
>>v=v(:);
```

```
>>Z=[ones(size(v)) v v.^2]
```

Z =

1	10	100
1	20	400
1	30	900
1	40	1600
1	50	2500
1	60	3600
1	70	4900
1	80	6400

Example: general least squares

Determine the coefficient matrix:

```
>>Z' * Z
```

```
ans =
```

```
      8      360      20400
     360     20400     1296000
    20400    1296000    87720000
```

$$\begin{bmatrix} n & \sum x_i & \sum x_i^2 \\ \sum x_i & \sum x_i^2 & \sum x_i^3 \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 \end{bmatrix} \begin{Bmatrix} a_0 \\ a_1 \\ a_2 \end{Bmatrix} = \begin{Bmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \end{Bmatrix}$$

```
>>F=F ( : ) ;
```

```
>>Z' * F
```

```
ans =
```

```
    5135
   312850
  20516500
```

```
>>a=(Z' * Z) \ (Z' * F)
```

```
a =
   -178.4821
    16.1220
     0.0372
```

`polyfit` and `polyval` function

MATLAB has a built-in function `polyfit` that fits a least-squares n th order polynomial to data:

```
p = polyfit(x, y, n)
```

x : independent data

y : dependent data

n : order of polynomial to fit

p : coefficients of polynomial $f(x)=p_1x^n+p_2x^{n-1}+\dots+p_nx+p_{n+1}$

MATLAB's `polyval` command can be used to compute a value using the coefficients.

```
y = polyval(p, x)
```

Exercise: Modify your `linregr.m` function to fit polynomials of any order using the built-in functions

Nonlinear regression

Not all fits are linear equations of coefficients and basis functions.

Problems in transformation:

- Not all equations can be transformed easily or at all
- The best fit line represents the best fit for the transformed variables, not the original variables.

Alternate approach: nonlinear regression.

Methodology:

- Write down the basis function.
- use MATLAB's **fminsearch** function to find the values of the coefficients where a minimum occurs

Example: nonlinear regression

For the wind tunnel data, determine the coefficients for the fit:

$$F = a_1 v^{a_2}$$

The m-file function:

```
function f = fSSR(a, xm, ym)
yp = a(1)*xm.^a(2);
f = sum((ym-yp).^2);
```

Then, use `fminsearch` to obtain the values of `a` that minimize `fSSR`:

```
a = fminsearch(@fSSR, [1, 1], [], v, F)
```

```
a =
```

```
2.5384
```

```
1.4359
```

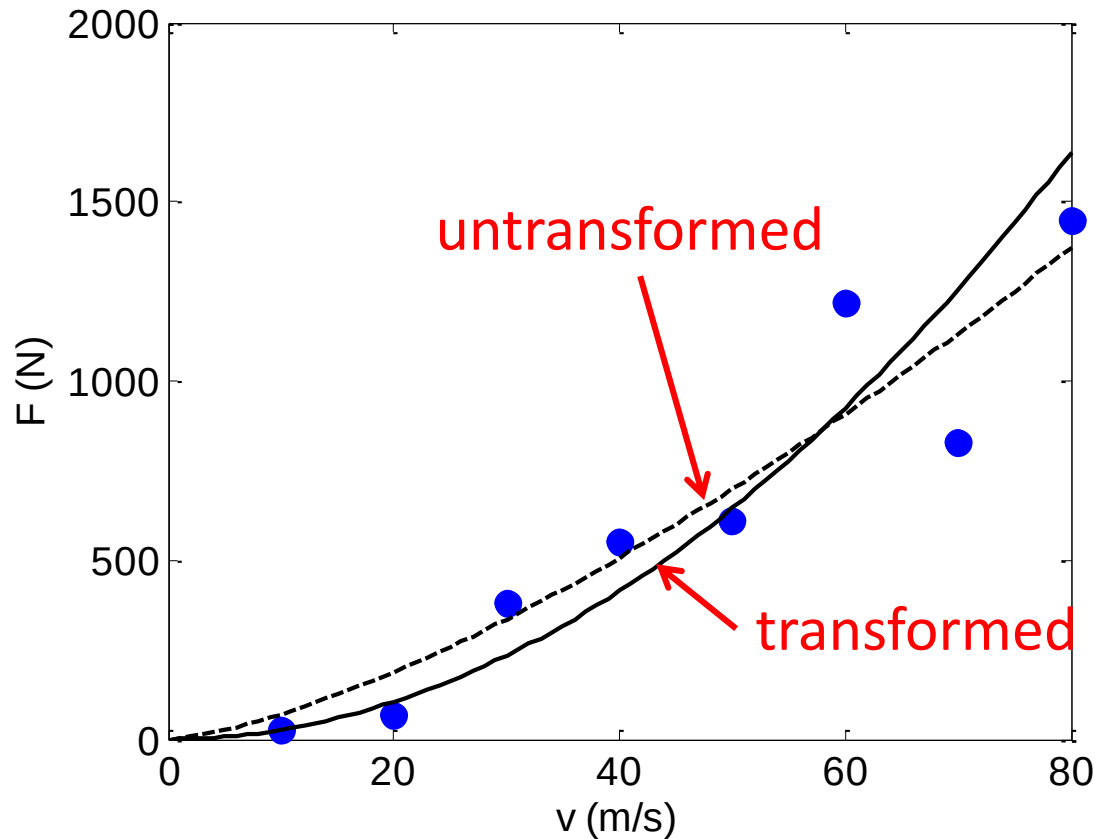
Initial guess for a1 and a2

*Option
placeholder*

Data

Comparison between regressions

$$F = a_1 v^{a_2} \begin{cases} \rightarrow F = 0.2741 v^{1.9842} & \text{transformed} \\ \rightarrow F = 2.5384 v^{1.4359} & \text{untransformed} \end{cases}$$



Polynomial interpolation

- (1) Use MATLAB's **polyfit** and **polyval** function
- the number of data points should be equal to the number of coefficients

- (2) Newton's interpolating polynomial

$$f_{n-1}(x) = b_1 + b_2(x - x_1) + \cdots + b_n(x - x_1)(x - x_2) \cdots (x - x_{n-1})$$

$$b_1 = f(x_1)$$

$$b_2 = f[x_2, x_1]$$

$$b_3 = f[x_3, x_2, x_1]$$

$$\vdots$$

$$b_n = f[x_n, x_{n-1}, \cdots, x_2, x_1]$$

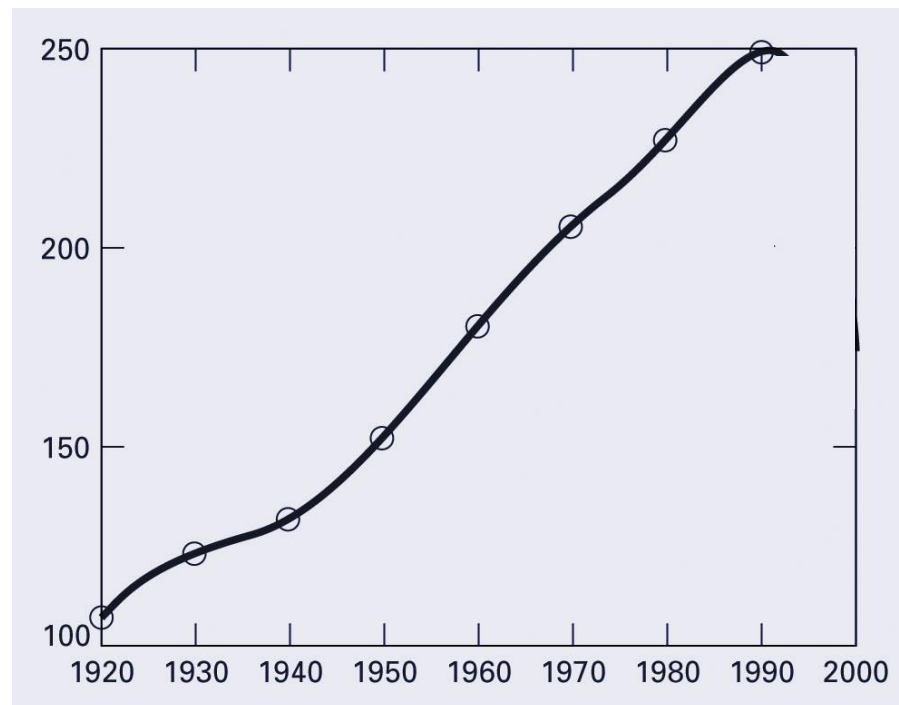
Divided
Differences

- (3) Lagrange's interpolating polynomial

Polynomial interpolation

Use **polyfit** and **polyval** function to predict the population of 2000 using a 7th order polynomial fitted with the first 8 points

Year	1920	1930	1940	1950	1960	1970	1980	1990	2000
Population (mil)	106.46	123.08	132.12	152.27	180.67	205.05	227.23	249.46	281.42



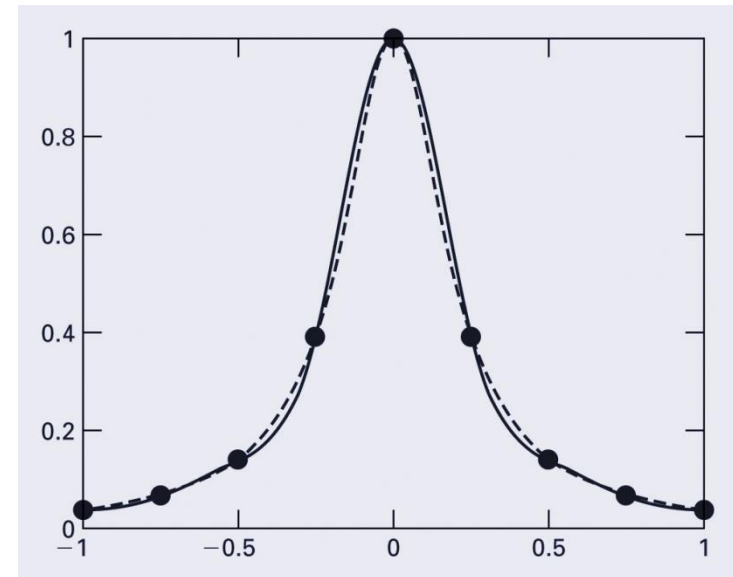
Piecewise interpolation in MATLAB

The spline command performs cubic interpolation, generally using not-a-knot method

```
yy=spline(x, y, xx)
```

Example: interpolate the Runge's function $f(x) = 1/(1+25x^2)$ taking 9 equidistant points between -1 and +1 using the spline command.

```
x = linspace(-1, 1, 9);  
y = 1./(1+25*x.^2);  
xx = linspace(0, 100);  
yy = spline(x, y, xx);  
yr = 1./(1+25*xx.^2)  
plot(x, y, 'o', xx, yy, '-', xx, yr, '--')
```



MATLAB's `interp1` function

MATLAB's `interp1` function can perform several different kinds of interpolation:

```
yi = interp1(x, y, xi, 'method')
```

`x` and `y` contain the original data

`xi` contains the points at which to interpolate

`'method'` is a string containing the desired method:

- `'nearest'` - nearest neighbor interpolation

- `'linear'` - connects the points with straight lines

- `'spline'` - not-a-knot cubic spline interpolation

- `'pchip'` or `'cubic'` - piecewise cubic Hermite interpolation

Example: `interp1` function

Time series of spot measurements of velocity:

t	0	20	40	56	68	80	84	96	104
v	0	20	20	38	80	80	100	100	125

```
>> tt=linspace(0,110);  
>> v1=interp1(t,v,tt);  
>> plot(t,v,'o',tt,v1)
```

